

ASSESSMENT TOOL 2
ITA0609 - Machine Learning

Sameer Ahmed G

192421154

Question 1

Recommendation of Data Plans Using K-Nearest Neighbour and Case-Based Learning

Introduction

Telecommunication companies offer multiple data plans with different data limits, prices, validity periods, calling benefits, and additional services. However, selecting the most suitable plan for every customer can be difficult because customers have different usage patterns. A customer who regularly streams videos may require a high-data plan, while another customer who mainly uses messaging and browsing may require only a small data plan. Machine Learning can analyze historical customer behavior and recommend the most appropriate plan automatically.

A suitable solution is to use the **K-Nearest Neighbour (KNN)** algorithm because it recommends a plan by comparing a customer's current usage pattern with similar customers from historical data. **Case-Based Learning (CBL)** can also be used because it stores previous customer cases and retrieves similar cases when a new customer requires a recommendation. In this case study, KNN is selected as the primary approach because it is simple, interpretable, and effective when customer behavior is represented using numerical usage features.

Problem Statement

The objective is to develop a recommendation system that analyzes customer usage patterns and recommends the most suitable telecommunications data plan. The system should consider parameters such as monthly data consumption, video streaming usage, browsing time, number of connected devices, average monthly spending, and peak-hour usage. It should also adapt when customer behavior changes over time.

Customer Features

The system can collect the following features:

Feature	Description
Monthly Data Usage	Total GB consumed per month
Video Streaming	Hours of video streaming per month
Browsing Time	Hours spent browsing
Connected Devices	Number of devices using the connection
Monthly Spending	Average monthly telecom expenditure
Peak Usage	Data consumed during peak hours

The target variable is the **recommended data plan**.

For example:

Customer	Data Usage	Streaming	Devices	Spending	Recommended Plan
C1	5 GB	10 hrs	1	₹199	Basic
C2	25 GB	45 hrs	2	₹399	Standard
C3	60 GB	100 hrs	4	₹699	Premium
C4	40 GB	70 hrs	3	₹599	Premium

Why K-Nearest Neighbour?

KNN is an instance-based learning algorithm. Instead of creating a complex mathematical model during training, it stores historical observations and compares a new customer with previously observed customers.

The fundamental idea is:

Similar customers are likely to prefer similar data plans.

For example, if a new customer consumes 55 GB per month, streams videos for 90 hours, and uses four devices, KNN searches the historical dataset for customers with similar usage patterns. If most of the nearest customers use the Premium plan, the system recommends the Premium plan.

Similarity Measurement

The most commonly used similarity measure for numerical features is **Euclidean distance**.

For two customers with feature vectors and ,

For example, suppose two customers have simplified feature vectors:

Customer A:

Customer B:

The Euclidean distance is

A smaller distance indicates greater similarity between customers.

Importance of Feature Scaling

Feature scaling is extremely important in KNN because distance calculations are affected by the numerical range of each feature. For example, monthly spending may range from ₹100 to ₹2,000, while the number of connected devices may range only from 1 to 5. Without scaling, spending would dominate the distance calculation.

Therefore, numerical attributes should be normalized using techniques such as **Min-Max Scaling**:

After scaling, all features have comparable influence on similarity calculations.

Selection of K

The value of represents the number of nearest customers considered when making a recommendation.

For example, if

the system finds the five most similar customers.

Suppose the five nearest customers have the following plans:

Nearest Customer	Plan
1	Premium
2	Premium
3	Standard
4	Premium
5	Premium

Premium receives four votes while Standard receives one vote.

Therefore,

The system recommends the **Premium Plan**.

An appropriate value of k can be selected using validation accuracy or cross-validation. A very small k can make the model sensitive to noise, while a very large k may ignore important local patterns.

Recommendation Generation

The recommendation process can be represented as:

Customer Usage Data



Data Preprocessing



Feature Scaling



Calculate Similarity



Find K Nearest Customers



Compare Their Data Plans



Majority Voting



Recommended Data Plan

The recommendation can also consider price and customer satisfaction. Therefore, instead of simply selecting the most common plan among neighbors, the system can rank plans according to predicted suitability, cost, and expected customer satisfaction.

Case-Based Learning Approach

Case-Based Learning provides an alternative solution. In CBL, every previous customer interaction is stored as a case containing the customer's usage characteristics, selected plan, and outcome.

For example:

Case:

Data Usage = 50 GB

Streaming = 80 hours

Devices = 3

Plan = Premium

Customer Satisfaction = High

When a new customer is encountered, the system retrieves similar historical cases and uses their outcomes to recommend a plan.

The CBL cycle consists of:

1. Retrieve similar previous cases.
2. Reuse the solution from those cases.
3. Revise the solution if necessary.
4. Retain the new experience for future recommendations.

This makes the system capable of learning continuously from customer interactions.

Adapting to Changing Customer Behaviour

Customer usage patterns can change significantly over time. For example, a customer may start working from home and increase their monthly data consumption. A static recommendation system may continue suggesting an outdated plan.

To address this issue, the system can maintain a **recent usage window**. Recent customer behaviour can be given greater importance than old records. For example, the system may calculate usage statistics for the most recent three or six months.

A time-weighted distance can also be used so that recent cases have greater influence.

The system can periodically retrain or update its customer database when new usage information becomes available. If a customer consistently exceeds the limits of their current plan, the system can automatically recommend an upgrade.

Evaluation of the Recommendation System

The recommendation system can be evaluated using:

- Recommendation Accuracy
- Precision
- Recall
- F1-Score

- Customer Satisfaction
- Plan Upgrade Rate
- Customer Retention
- Reduction in Plan Downgrades

For example, if 1,000 recommendations are generated and 900 customers select or successfully use the recommended plan, recommendation accuracy is approximately:

Customer feedback can also be incorporated to determine whether recommendations are genuinely useful rather than simply matching historical behavior.

Advantages

KNN is easy to understand and implement, does not require complex model assumptions, naturally captures local customer behavior, and can be updated by adding new customer records. Case-Based Learning similarly provides explainable recommendations because the system can show previous customers with similar usage patterns.

Challenges

The main challenges include selecting suitable features, handling large customer databases, determining an appropriate value of k , scaling numerical features, dealing with noisy usage data, and adapting to rapidly changing customer behavior. KNN can also become computationally expensive when the number of customers becomes very large because distances must be calculated against many historical records.

Conclusion

A KNN-based recommendation system provides an effective solution for recommending telecommunications data plans based on customer usage behavior. Similarity between customers can be measured using Euclidean distance after appropriate feature scaling, and the plans used by the nearest customers can be combined through majority voting to generate recommendations. Case-Based Learning can further improve the system by storing previous customer experiences and retrieving similar cases. By continuously incorporating recent usage patterns, the system can adapt when customers change their behavior. Therefore, KNN and Case-Based Learning provide practical, interpretable, and adaptive approaches for personalized telecommunications plan recommendations.

Question 2

Energy Consumption Prediction Using Locally Weighted Regression

Introduction

Electricity consumption varies according to several factors, including time of day, weather conditions, number of occupants, appliance usage, working hours, weekends, and seasonal changes. A conventional global regression model attempts to learn one relationship between these variables and electricity consumption for the entire dataset. However, electricity consumption does not always follow a single global pattern. For example, consumption behavior during summer afternoons can be very different from consumption during winter nights. A **Locally Weighted Regression (LWR)** model is suitable for this problem because it gives greater importance to observations that are similar to the current input and lower importance to distant observations.

LWR is an instance-based local learning method. Instead of assuming that one global regression equation accurately describes every situation, it constructs a local model around the point being predicted. This allows the model to capture variations in electricity consumption caused by weather, time, and appliance usage.

Problem Statement

An energy management company wants to predict electricity consumption using factors such as time, weather, temperature, humidity, appliance usage, and historical consumption. The system must capture local consumption patterns, handle noisy measurements, account for seasonal variations, and provide accurate predictions that can support energy planning and demand management.

Input Features

The proposed model can use the following attributes:

Feature	Description
Hour	Time of electricity consumption
Day Type	Weekday or weekend
Temperature	Outdoor temperature
Humidity	Atmospheric humidity
Appliance Usage	Number/intensity of active appliances
Previous Consumption	Recent electricity consumption
Season	Summer, Winter, Monsoon, etc.
Target	Electricity consumption in kWh

A sample dataset can be represented as:

Time	Temperature	Humidity	Appliance Usage	Previous Consumption	Consumption
08:00	27°C	65%	4	3.2 kWh	3.5 kWh
12:00	32°C	55%	7	4.8 kWh	5.6 kWh
18:00	30°C	60%	8	5.5 kWh	6.2 kWh
23:00	26°C	70%	3	3.0 kWh	2.8 kWh

Why Locally Weighted Regression?

Traditional linear regression estimates a single global relationship:

This may not represent electricity consumption accurately because consumption patterns change depending on time, temperature, and season.

LWR instead assigns weights to training observations according to their similarity to the query point. Nearby observations receive higher weights, while distant observations receive lower weights.

Therefore, LWR can model different consumption patterns for different conditions.

Similarity and Weight Calculation

The distance between the current observation and a training observation can be calculated using Euclidean distance.

A Gaussian weighting function can then be used:

where:

- = distance between observations
- = weight assigned to observation
- = bandwidth parameter

A nearby observation receives a high weight, while a distant observation receives a low weight.

Example of Local Learning

Suppose the company wants to predict electricity consumption at:

Temperature = 32°C

Time = 14:00

Appliance Usage = High

The algorithm searches historical records with similar temperature, time, and appliance usage.

Suppose the nearest observations are:

Historical Observation	Consumption	Similarity
A	5.8 kWh	Very High
B	6.1 kWh	High
C	5.5 kWh	High
D	4.0 kWh	Low

Observations A, B, and C receive larger weights because their conditions are similar to the current situation. Observation D receives a smaller weight because its conditions are significantly different.

The weighted observations are then used to calculate the predicted consumption.

Working of LWR

The complete process is:

Historical Energy Data



Data Preprocessing



Feature Scaling



New Energy Condition



Calculate Distance



Assign Local Weights



Fit Local Regression Model



Predict Consumption



Evaluate Prediction

Handling Noisy Data

Real-world energy datasets contain noise due to sensor errors, sudden appliance switching, communication failures, and unexpected changes in electricity usage.

LWR naturally reduces the influence of observations that are far from the current condition. Additional preprocessing can further improve reliability.

Noise-handling techniques include:

- Removing duplicate records
- Handling missing values
- Outlier detection
- Moving-average smoothing
- Median filtering
- Feature normalization

Outliers can also be detected using the Interquartile Range (IQR) method or Z-score.

Handling Seasonal Data

Electricity consumption varies considerably across seasons. During summer, air-conditioning usage may increase electricity demand, while winter may produce different heating patterns.

The system can include **season** as an input feature. Historical data can also be divided into seasonal subsets.

For example:

Summer → High cooling demand

Winter → Moderate demand

Monsoon → Variable demand

LWR is particularly useful because it gives more importance to historical observations that resemble the current seasonal and weather conditions.

Time-Based Patterns

Electricity consumption also changes throughout the day.

Typical patterns may include:

Early Morning → Low Consumption

Morning → Increasing Consumption

Afternoon → High Consumption

Evening → Peak Consumption

Night → Decreasing Consumption

LWR can capture these local variations because predictions are based on observations that are similar to the current time and conditions.

Selection of Bandwidth

The bandwidth parameter controls how local the regression model is.

If is too small, only very close observations receive significant weights. This may cause the model to overfit noise.

If is too large, distant observations receive significant weights and the model becomes similar to global regression.

Therefore, the bandwidth should be selected using validation data or cross-validation.

Model Evaluation

The model can be evaluated using regression metrics such as:

Mean Absolute Error

Mean Squared Error

Root Mean Squared Error

R² Score

Lower MAE and RMSE indicate better prediction performance, while a higher R² indicates that the model explains a larger proportion of the variation in electricity consumption.

Example Evaluation

Suppose the LWR model produces:

Metric	Result
MAE	0.42 kWh
RMSE	0.61 kWh
R ²	0.93

An R² value of 0.93 indicates that approximately 93% of the variation in electricity consumption is explained by the model.

Alternative Approach: RBF Networks

An alternative solution is a **Radial Basis Function (RBF) Network**. RBF networks use radial basis functions to measure the similarity between an input and learned centers.

A common Gaussian radial basis function is:

where μ represents the center and controls the width of the function.

RBF networks are useful for nonlinear electricity consumption patterns because they can model complex relationships between temperature, time, appliance usage, and demand.

For this case study, LWR is preferred because it provides an intuitive local-learning mechanism and directly explains why similar historical observations receive greater influence.

Applications

The proposed system can be used for:

- Smart grid management
- Household energy forecasting
- Industrial energy management
- Peak load prediction
- Renewable energy planning
- Electricity pricing
- Demand-response systems
- Energy-efficient building management

Advantages

Locally Weighted Regression can capture nonlinear and local relationships, adapt to changing consumption patterns, give greater importance to similar historical observations, and provide interpretable predictions. It is particularly useful when the relationship between input variables and electricity consumption changes across different regions of the feature space.

Challenges

LWR can become computationally expensive for very large datasets because distances and weights must be calculated for each prediction. Selecting an appropriate bandwidth is also important. Very small bandwidths may lead to overfitting, while very large bandwidths may fail to capture local behavior. Missing sensor data, unexpected weather events, sudden appliance usage, and long-term changes in consumer behavior can also reduce prediction accuracy.

Conclusion

Locally Weighted Regression provides an effective solution for predicting electricity consumption because it learns locally rather than assuming one global relationship across all observations. By assigning greater weights to historical observations that have similar time, weather, and appliance-usage conditions, the model can capture variations in electricity demand more effectively. Data preprocessing, normalization, outlier handling, and seasonal features improve robustness against noisy and changing data.